

Segurança da informação

Nome dos integrantes: Gabriel Vitório, Lucas Leal de Oliveira, Yuri Arins

Turma: 5A

Data: 11/05/2026

Link do repositório:

<https://github.com/Lucas-Leal-Oliveira05/Seguranca-sistema-academico>

O sistema que nos foi fornecido é um protótipo web simples para registro de ocorrências acadêmicas.

Suas principais funcionalidades são:

- Criar ocorrências
- Pesquisar registros
- Mostrar ocorrências cadastradas
- Exportar todos os Dados
- Restaurar dados iniciais
- Limpar Logs

No sistema temos os perfis de usuário que são o Administrador, Professor e Aluno.

A aplicação manipula dados de alunos, administrador e professor. Entre os dados manipulados estão nome do aluno, Matrícula, CPF, e-mail pessoal, telefone, tipo de ocorrência, descrição da ocorrência e observações internas.

O sistema armazena tudo no próprio navegador pois não possui banco de dados real nem API real e back-end

• Quem pode cadastrar ocorrências?

Todos os perfis podem cadastrar as ocorrências

• Quem pode visualizar registros?

Todos os usuários podem visualizar os registros de ocorrências e logs

• Quem pode alterar ou excluir informações?

Todos os perfis tem a permissão para alterar e excluir as informações

• Quais ações deveriam ser restritas a determinados perfis?

- Visualização de ocorrências: professores e administradores
- Visualização de logs: apenas administradores
- Exclusão de logs: apenas administradores

- Alterar status da ocorrência: apenas administradores
- Excluir ocorrências: apenas administradores
- Exportar dados: apenas administradores

• **Há diferença real entre os perfis de usuário?**

Não existe uma diferença entre os perfis. Todos os perfis possuem um mesmo nível de acesso, ou seja, não existe o controle de acesso e de permissões no sistema. Além disso, o próprio usuário pode alterar seu tipo de perfil, alternando entre aluno, professor ou administrador quando bem quiser

• **Há alguma regra importante que está apenas simulada no front-end?**

A autenticação de login e a lógica de permissões

Lista de ativos:

Item	Ativo identificado	Por que tem valor?
1	Interface do sistema	Apresenta informações para usuários
2	Código-fonte	Possui a estrutura e regras do sistema inteiro
3	Logs	Registram ações ocorridas no sistema
4	Perfis de usuários	Permite que cada usuário acesse o sistema
5	Dados de usuários	Informações pessoais utilizadas no sistema
6	Ocorrências cadastradas	Registro das ocorrências já
7	Dados exportados	Apresenta informações de tudo que foi registrado até o momento da exportação
8	Token de API	Permite acesso à API

Item	Dados ou Ativo	Classificação	Justificativa
1	Interface do Sistema	Interno	Ela faz parte do funcionamento do sistema, e é utilizada pelos usuários e administrados para acessar algumas funcionalidades do sistema
2	Código Fonte	Confidencial	Contém toda logica e funcionamento do sistema.
3	Logs	Confidencial	Contém apenas todos os registros eventos e ações do sistema.
4	Perfis de usuários	Confidencial	Define permissões e níveis de acesso dentro do sistema
5	Dados de Usuários	Confidencial	Contêm informações pessoais que precisam de maior proteção
6	Ocorrências Cadastradas	Interno	Tem registros importante utilizados pela aplicação para controle das ocorrências
7	Dados Exportados	Confidencial	Contém informações importantes da aplicação e dos usuários
8	Token API	Restrito	Qualquer pessoa com o token tem livre acesso à API

Análise de riscos

Item	Problema	Risco	Possível impacto
1	Seleção de tipo de conta	Usuários podem garantir a	Usuários podem realizar ações na qual não deveriam

		permissão que quiserem	
2	Limpeza de logs	Exclusão não intencional dos logs	Exclusão total dos logs registrados
3	Seção de log	Todos os usuários têm acesso aos logs	Usuários obterem acesso a informações que não deveriam
4	Campos de ocorrências com poucas restrições	Dados de tipo incorretos ou vazios sendo cadastrados	
5	Falta de controle de sessão	Maiores chances de alguma pessoa acessar conta pertencente à outra	Ações indesejadas dentro do sistema + possível vazamento de informações
6	Senhas visíveis	Pessoa não autorizada acessa conta de outro usuário	Ações indesejadas dentro do sistema + possível vazamento de informações
7	Token de API visível	Usuários possuem fácil acesso ao token (segredo)	Uso indevido de API

Riscos e melhorias do código:

item	Problema	Melhoria proposta	Implementado?	Justificativa
1	Injeção de código (XSS)	Trocar o uso de 'innerHTML' por 'textContent' além de implementar uma sanitização dos dados de entrada pelos usuários	Parcialmente implementado	'innerHTML' interpreta HTML e scripts enviados pelo usuário. A troca e a sanitização visam evitar que o sistema rode scripts maliciosos pelos usuários
2	Controle de permissões	Remover troca de nível de perfil manual e restringir algumas ações para apenas admins utilizarem. Implementar uma arquitetura RBAC	Parcialmente implementado	Atualmente qualquer usuário poderia virar admin quando bem quiser. A melhoria visa que funções e informações sensíveis não possam ser executadas e visualizadas por usuários além de admins
3	Log com dados sensíveis	Retirar informações sensíveis e deixar apenas informações cruciais	implementado	Deixar o vazamento de logs menos perigosos, além de facilitar auditorias e

				estar mais de acordo com a LGPD
4	Dados sensíveis expostos	Mascarar informações como cpf e telefone	Parcialmente implementado	Minimizar os riscos de vazamento de dados sensíveis e acessos indevidos
5	Validação de entradas	Implementar validação das entradas dos usuários (tamanho, formato e caracteres)	Implementado	Ausência destas validações pode permitir a inserção de scripts. A mudança ajuda a proteger o sistema de inserção dos scripts e possíveis destruições de dados
6	Funções perigosas sem restrição	Restringir funções de exclusão, limpeza e restauração dos logs para apenas admins utilizarem. Além disso, adicionar uma confirmação para ativar	Parcialmente implementado	Por hora, qualquer usuário poderia “destruir” as informações do sistema. As alterações visam reduzir alterações indevidas ou modificações acidentais
7	Token exposto	Remover tokens do front e armazenar em local seguro	Implementado	Qualquer usuário poderia utilizar o token, causando em utilizações indevidas da API. Mudança visa dificultar e impedir o acesso indevido da API
8	Falta de HTTPS	Quando em produção, realizar a publicação via HTTPS	Depende de infra	O uso do HTTPS garante que dados trafeguem com criptografia, impedindo a interceptação
9	Senhas totalmente expostas	Remover credenciais de acesso do front (em texto puro). Além disso, implementar autenticação com hash das senhas	Não implementado (intencional por motivos didáticos) / Depende de back	Tudo em texto puro pode ser acessado por qualquer usuário. A remoção das credenciais do front e o uso de hash protege as senhas, impedindo vazamentos de dados e acessos indevidos
10	LocalStorage desprotegido	Trocar o uso de localStorage para sessionStorage. Além disso, utilizar de cookies no back	Parcialmente implementado	LocalStorage pode ser modificado via scripts. A mudança visa evitar roubo de informações, de sessão e alterações de dados

Resumo das modificações realizadas:

1. Injeção de código (XSS)
Foi implementado uma sanitização de dados através da função “escapaHTML()”. Ela valida os dados da entrada do usuário e retira/modifica partes potencialmente perigosas para o sistema, assim, garantindo uma maior integridade.
2. Restrição de acesso à funções de adm
Foi implementado a função “verificaAdm()”, que verifica se o perfil do usuário é um perfil de adm. Além disso, as funções de exclusão, exportação, limpeza de logs e restauração de dados agora passam por esta verificação, sendo executado apenas por pessoas autorizadas. Essa modificação se associa ao conceito do menor privilégio. A arquitetura RBAC não foi implementada pois é dependente de um back.
3. Remoção da alteração de perfil
Agora usuários não podem alterar o tipo da própria conta (entre aluno, professor ou adm). Essa modificação evita o escalonamento de privilégios e garante uma maior integridade do sistema
4. Informações sensíveis dos logs
Logs não armazenam mais CPF, descrição e objetos inteiros. Assim, garante uma maior confidencialidade, se adequa parcialmente à LGPD e previne um vazamento de dados sensíveis
5. Informações sensíveis expostas
Foi implementado a função mascarar cpf que esconde o dado afim de garantir maior privacidade
6. Validação de entrada de usuário
Foi implementando uma validação dos dados de entrada do usuário, validando tamanho, formato de cpf, email e limite de caracteres. Essa função visa aumentar a segurança contra injeção de código e garantir integridade de dados
7. Proteção de funções sensíveis
Foi implementado uma confirmação ao tentar utilizar funções potencialmente destrutivas.
8. Remoção do token
O token foi removido do código afim de evitar que pessoas indevidas possam se utilizar indevidamente da API (esse código não possui a API funcionando)

9. Troca do localStorage

LocalStorage foi trocado por sessionStorage para fazer os dados durarem apenas durante a sessão. Essa modificação visa retirar a persistência de dados sensíveis e é um começo de gerenciamento de sessão